PQ-GATE — Project Report (end to end)

Passing Tests, Failing Security: Measuring and Gating LLM-Driven Post-Quantum Migrations

Target venue: ICSE 2027 workshop (deadline 13 Nov 2026). Report date: 23 Oct 2026. Status: all experiments complete (Phases 0–4); paper writing remains.

---

1. Summary in one paragraph

We asked whether code that AI models migrate from classical cryptography (RSA, elliptic curves) to post-quantum cryptography (ML-KEM, ML-DSA) can pass all of its tests and still be insecure. We built a benchmark of 20 realistic Python programs with 77 deliberately broken but test-passing migrations, and a security gate (static rules, a dataflow checker and behavioural attacks) that catches all 77 while the tests and standard tools catch almost none. We then had four AI models perform 400 real migrations. Of the 84 that migrated and passed every test, the gate flagged 26 as insecure (31 before correcting a gate bug), while Semgrep and Bandit flagged 0 and 1. A blind, independent check estimated that the gate catches about 63% of the real flaws and is right about 5 in 6 times it raises an alarm on real migrations. Finally, when models were asked to repair their insecure code, test feedback fixed 0 of 31 cases, while gate feedback let the strongest model genuinely fix 10 of its 15 flawed files.

---

2. Why this project

- **The migration is coming.** Large quantum computers would break RSA and elliptic-curve cryptography. NIST has standardised replacements (ML-KEM, FIPS 203, for key exchange; ML-DSA, FIPS 204, for signatures), and organisations must migrate a great deal of code.
- **AI will do much of that migration.** Developers increasingly ask LLMs to rewrite code, and then check the result by running the existing tests.
- **Tests check behaviour, not security.** A migration can encrypt and decrypt correctly (tests pass) while, for example, using the raw KEM secret as a key, keeping a classical fallback, leaking keys into logs, or signing the wrong data. These flaws are invisible to functional tests.
- **Gap:** there is no benchmark that measures this, and no tool aimed at it. General security linters (Bandit, default Semgrep) know nothing about post-quantum migration mistakes.

3. Research questions

1. **RQ1 (prevalence):** How often do AI-made PQC migrations pass all tests but remain insecure?
2. **RQ2 (detection):** Can a PQC-aware gate detect these flaws better than tests and general-purpose tools?
3. **RQ3 (repair):** Does gate feedback help a model fix the flaws, compared with test feedback?

4. Ground rules (from the project brief)

1. Never fabricate results: every number comes from a script that actually ran; failures are recorded as failures.
2. The builder (Claude Code) is not a test subject: counted migrations come only from the evaluated AI models.
3. No hand-written crypto primitives: `cryptography` (pyca) for classical, `liboqs-python` for ML-KEM / ML-DSA.
4. Reproducibility: prompts, temperature, seeds and sample counts fixed in one config file; every raw output saved; model name, tag and size logged.
5. Python only, simple code. Stop at each checkpoint. Keep `PROGRESS.md` up to date. Low disk use.

---

5. Timeline (when)

Date (2026)	What happened
12 Oct	Phase 0: environment, liboqs, Ollama models, config; smoke tests; the "API reference card" for prompts decided before any model run.
13–14 Oct	Phase 1: 20 benchmark tasks with 71 broken variants, all test-invisible (Checkpoints 1a, 1b).
14 Oct	Phase 2: the gate built and validated (Checkpoint 2); gate frozen; 6 held-out variants tested once; Phase 3 started.
15 Oct	Low yield from gemma seen in a preview; qwen2.5-coder:7b added.
15–18 Oct	Interruptions: session ends, a laptop shutdown, battery death; an automatic Ollama update (0.40.3 → 0.40.6). Each handled and logged.
19 Oct	gpt-oss-120b (via Groq API) added as the large-model condition.
20 Oct	All 400 migrations finished and scored; blind labelling set built.
21 Oct	Independent AI labelling and gate comparison (Checkpoint 3); Phase 4 repair loop started.
22 Oct	Phase 4 finished; all gate-guided fixes hand-audited; A2 naming bug found.
23 Oct	This report.

---

6. Setup (Phase 0)

Item	Detail
Machine	Windows 10, Intel i5-6300U (2 cores / 4 threads), 16 GB RAM, no GPU (models run on CPU)
Python	3.11 virtual environment `pq-gate/.venv`; pinned packages in `requirements.txt`
Crypto	`cryptography` 50.0.2; `liboqs-python` 0.16.0.1 with the prebuilt liboqs 0.16.0 Windows DLL (MSYS2 package; source URL and SHA-256 in `README.md`)
Tools	Semgrep 1.180.0, Bandit 1.9.4, pytest 9.1.1
Local models (Ollama)	gemma3:4b (digest c30276f7dffe), qwen2.5-coder:3b (f72c60cabf62), qwen2.5-coder:7b; models stored on `E:\OllamaModels`
API model	openai/gpt-oss-120b through Groq (free tier; key in `.env`, git-ignored, never logged)
Generation	temperature 0.2, 5 samples per task with seeds 1001–1005, context 8192, output cap 3072 tokens (8192 for gpt-oss, which also spends tokens on reasoning)
Prompt	Fixed instruction ("Migrate this code ... using liboqs (ML-KEM / ML-DSA). Keep the same public functions so existing tests still pass. Return only the full Python file.") plus a neutral API reference card (how to call ML-KEM, ML-DSA, HKDF; no security advice) plus the original file

Why the API card: in smoke tests the small models did not even know what ML-KEM is, so without API docs almost nothing would run. Real developers read docs too. The card shows API usage only; it deliberately contains no advice such as "use a KDF", so it does not teach the answers the gate checks for. Threat to validity: showing the HKDF API may make F1 less likely than with no card.

---

7. Phase 1 — The benchmark

7.1 How it is built

Each of the 20 tasks lives in `benchmark/task_NN_name/` and contains:

File	Purpose
`original.py`	The classical program (55–107 lines), the input the models migrate
`reference_pqc.py`	One correct post-quantum migration
`test_original.py`	Functional tests (round trips, tampering, wrong keys); loads the file under test from the `PQGATE_TARGET` environment variable so the same tests run on every version
`variants.py`	Each broken variant as a list of exact text edits applied to the reference (so each is a small, auditable one-flaw change)
`broken/*.py`	The generated broken variants
`meta.yaml`	Task description, and for each variant its flaw type and the gate checks expected to catch it

7.2 The 20 tasks (5 families)

Family	Tasks
1 Key wrapping (RSA-OAEP → ML-KEM + HKDF + AES-GCM)	01 file key wrap, 04 multi-recipient backup, 05 password vault, 06 sealed deployment secrets
2 Session setup (X25519 → hybrid X25519 + ML-KEM)	02 session handshake, 07 device pairing, 08 tunnel with rekeying, 09 sealed messages
3 Document signing (RSA-PSS/ECDSA → ML-DSA-65)	03 document signing, 10 update manifest, 11 invoices, 12 audit log
4 Token signing (RS256/ECDSA → ML-DSA-65)	13 API token, 14 session cookie, 15 license key, 16 webhook signature
5 Config-chosen algorithm	17 provider registry, 18 signer negotiation, 19 policy string, 20 cipher-suite negotiation

7.3 Flaw taxonomy (F1–F10)

ID	Flaw	Variants
F1	Raw KEM shared secret used directly as the AES key (no KDF)	6
F2	Hybrid combiner without binding (XOR, or no ciphertext/transcript in the KDF)	5
F3	Classical fallback still reachable on an exception or config path	15
F4	Hybrid where only the classical half decides	6
F5	Key material written to logs, exceptions or temp files	16
F6	Weak or fixed randomness (`random` module, fixed seed)	5
F7	ML-DSA context ignored or wrong data signed	9
F8	Non-constant-time comparison of MACs/tags	6
F9	Downgrade through algorithm preference order	5
F10	AES-GCM nonce reuse	4
	Total	**77** (71 original + 6 held-out)

7.4 What was tested

- `scripts/validate_benchmark.py`: every original, reference and variant against its task's tests. **Result: all 20 originals and 20 references pass; 71/71 variants pass every test (test-invisible).** The 6 held-out variants added later also pass every test.
- Behavioural sanity checks (throwaway scripts, not results): all 33 flaws that show up at runtime fire on the variant and not on the reference.

---

8. Phase 2 — The security gate

8.1 Design (two tiers)

Tier 1, static (the code is not run):

Check	Meaning	Implementation
P1	KEM output must go through a KDF before becoming a cipher key	Semgrep rules + dataflow (raw secret reaches AESGCM without `.derive()`)
P2	No RSA / elliptic-curve / OAEP after migration	Semgrep rules
P3	No secrets in logs, print, exceptions or file writes	Semgrep rules (name-based)
P4	No `random` module	Semgrep rule
P5	MACs/tags compared with `hmac.compare_digest`, not `==`	Semgrep rule (name-based)
P6	Hybrid KDF takes both secrets and binds the KEM ciphertext	Dataflow checker (`gate/static_dataflow.py`)

Tier 2, adversarial (the code is run against attacks):

Check	Attack
A1	Make liboqs unavailable or failing: does the code silently fall back to classical crypto?
A2	Offer a classical option alone or next to the PQC one: is it accepted or preferred?
A3	Wrong/oversized keys and tampered ciphertext: does key material leak into exceptions, logs or output, or is tampered data accepted?
A4	Corrupt only the ML-KEM secret in a hybrid: do both sides still agree?
A5	Change the context, a signed field or the message tail after signing: does it still verify?
A6	Record every (key, nonce) pair used by AES-GCM: is any pair reused?

`gate/run_gate.py` combines both tiers into one verdict per file: PASS, FAIL (with the list of checks that fired) or ERROR (if the static tier could not run, so a crash is never reported as PASS). Targets run inside a throwaway temp folder.

8.2 Why P6 is a dataflow check, not a Semgrep rule

The first Semgrep version of P6 (and a name-based P1 rule) wrongly flagged 9 of the 20 correct references. The property "the KDF input must contain both secrets and its info must contain the ciphertext" is about something being absent along a data path, which Semgrep patterns cannot express. A small AST taint analysis (tags CL / PQ / CT) solved it with 0 false positives. Logged as a decision.

8.3 Validation on the benchmark (in-sample)

	Tests only	Default Semgrep (p/python)	Bandit	PQ-GATE
Variants flagged (of 71)	0	2	7	**71**
Correct references flagged (of 20)	0	0	0	**0**

Every one of the 71 is caught by a check listed for it in `meta.yaml` before the gate existed. **Caveat:** the gate was debugged on this same benchmark, so this shows the gate can express each flaw, not how accurate it is on unseen code.

8.4 Held-out test (out-of-sample)

The gate was frozen (hashes of all gate files saved), then 6 new variants were written in new shapes (3 × F4, 3 × F8), and the gate was run once. **Result: 4/6.** F4: 3/3 caught. F8: 1/3 (the name-based P5 rule missed `a.hex() == b.hex()` and a hand-written byte loop). Not tuned afterwards.

---

9. Phase 3 — 400 real AI migrations

9.1 Protocol

4 models × 20 tasks × 5 samples = 400 migrations (`harness/migrate.py`), every raw output and full log saved in `results/raw/`. Each output was then scored (`scripts/evaluate.py`): does it parse, does it import liboqs, do all tests pass, the gate verdict, and the default-Semgrep and Bandit findings. Model outputs are untrusted code, so tests and attacks ran in a subprocess with a timeout inside a throwaway folder.

9.2 Results

Model	Pass all tests	...and use liboqs	...and gate FAIL
gemma3:4b	71 / 100	5	5
qwen2.5-coder:3b	52 / 100	1	1
qwen2.5-coder:7b	9 / 100	9	5
gpt-oss-120b	69 / 100	69	20 (**15** after correction)
All	201 / 400	**84**	**31 (26 after correction)**

- **Baselines on the same 84:** default Semgrep flagged 0, Bandit flagged 1.
- **gpt-oss flaws found:** raw ML-KEM secret used as the AES key (P1, 9 files), token signatures without the context string (A5, 5), classical option still selectable (A2, 7 files, of which 5 are the false alarms explained in 9.3).
- **117 outputs pass every test while still being classical code** (mostly the small models returning the original nearly unchanged). The tests cannot even tell whether a migration happened; the gate flags all of them (P2).
- The 7B model attempts real migrations more often, so it breaks the tests more often (9/100 pass).
- Flags cluster by task (the 5 samples of one task look alike), so the effective number of independent cases is about 10 tasks, not 26 files.

9.3 Correction found later (A2 naming bug)

The A2 suite probe treats a cipher suite as post-quantum only if its name contains "MLKEM". gpt-oss named its suite `ML-KEM-768-AES256GCM` (with a hyphen), so all 5 of its task_20 outputs were false alarms. The frozen gate was not changed; both counts are reported (31 → 26 overall, 20 → 15 for gpt-oss).

9.4 Stricter "was it really migrated?" measure

The blind labeller found that importing liboqs is not proof of a migration (11 of 35 sampled files imported it but still used RSA/ECDSA). `scripts/strict_migration.py` counts a file as migrated only if liboqs is called inside a function; it agrees with the labeller on 33/35 files. Under it: 83 test-passing migrations, 30 gate FAIL (frozen gate).

---

10. Checkpoint 3 — Is the gate right? (blind labels)

- 35 files sampled at random (seed 2026) from the 84 test-passing liboqs outputs, at most 20 per model.
- Labelled by an **independent Claude agent** that saw only the labelling guide, the files and each task's original/reference code, and was barred from all gate results. (Chosen by the user over human labelling; a human spot-check of about 10 items is still recommended.)
- Labels: 16 secure, 19 insecure (F1 ×6, F7 ×2, F5 ×2, F3 ×1, and 11 not really migrated).

	Precision	**Recall**
All 35 files	16/17 = 94%	16/19 = 84%
Real migrations only (24 files)	**5/6 = 83%**	**5/8 = 63%**

Use the real-migration figures. The 84% is inflated by the 11 easy unmigrated files. The four disagreements:

File	**Who is right**	**Cause**
item_30 (missed F1)	labeller	the raw key leaves the module and comes back through a caller; the dataflow check does not follow that
item_15, item_31 (missed F7)	labeller	A5 has no probe for task_18's `seal_envelope` / `open_envelope`
item_18 (false alarm)	labeller	the A2 naming bug above

---

11. Phase 4 — Repair loop (gate feedback vs test feedback)

11.1 Protocol

- Items: the 31 outputs that pass all tests, use liboqs and fail the gate. Each is repaired by the model that wrote it, same seed.
- Two conditions with identical prompts except the feedback: **tests** (test results only, e.g. "all 10 tests passed") and **gate** (test results plus the gate's findings with line numbers).
- 1 to 3 rounds; stop when the condition's own signal is clean, or when the model returns the file unchanged.
- "Fixed" = the final file passes all tests and the gate. Every round saved under `results/repair/`.

11.2 Results

Model	**Tests feedback**	**Gate feedback**
gpt-oss-120b (20)	**0/20** fixed	**10/20** fixed (= 10 of the 15 genuinely flagged)
gemma3:4b (5)	0/5	0/5
qwen2.5-coder:7b (5)	0/5	0/5 (broke the tests in all 5)
qwen2.5-coder:3b (1)	0/1 (never changed)	0/1 (never changed)
All 31	**0/31**	**10/31**

- **Hand audit:** the gate both gives the hints and judges the result, so a model could cheat (for example by renaming a variable). All 10 fixes were checked by hand and **all are genuine**: 9 added an HKDF key derivation, and others actually removed the RSA provider or the classical key-exchange option.
- Not fixed by gpt-oss: the 5 token-signature context cases (task_13); 1 kept the flaw, 4 broke the tests. The 5 task_20 items "not fixed" were the A2 false alarms.
- Small models cannot act on the feedback: they ignore it, or (qwen-7B) break working code.

---

12. Limitations (to state in the paper)

1. In-sample gate validation (71/71) is optimistic; the honest estimates are the held-out 4/6 and the real-migration recall of 63%.
2. Gate bugs and gaps found: A2 suite-name bug (5 false alarms); A5 has no probe for one task's API; the dataflow check does not follow values through callers; P3 and P5 depend on variable names (one P3 false alarm on `len(shared_secret)`).
3. Labels come from an AI agent, not a human.
4. Few independent cases: flaws cluster in about 10 tasks.
5. Mixed Ollama versions: gemma ran under 0.40.3, the qwen models under 0.40.6 (identical model weights).
6. The benchmark's tests deliberately cover normal behaviour only; that is the premise, but it is a design choice.
7. The API card shows the HKDF and ML-DSA context APIs, which may make F1/F7 rarer than with no documentation.
8. Small, CPU-only local models; one large model through a free API tier.

13. Integrity and reproducibility measures

- Inputs (config, prompts, harness, originals) and the gate were hashed before each run (`results/frozen/`).
- The gate was never tuned on data used to report its accuracy; post-hoc corrections are reported next to the frozen numbers.
- Infrastructure failures (server errors, crashes, network outages, rate limits) were re-run; model failures (bad code, timeouts) were kept. Replaced samples were moved to `results/raw_superseded/`, never deleted.
- Every decision is logged with its date and reason in `PROGRESS.md`.

Incidents handled: background jobs dying with the session (fixed with detached watchdog scripts); laptop shutdown and battery death; Ollama auto-update mid-run (version split documented, 6 samples regenerated); a failed 7B download (cleaned and re-downloaded); Groq rate limits and network outages (stop without writing, resume later); a Windows command-line length limit (batched Semgrep/Bandit); several bugs in my own scripts, each fixed and logged.

---

14. Code map — what is where, and why

14.1 Configuration and documentation

Path	Lines	What it does	Why
`config.yaml`	100	Models, generation settings, seeds, the migration prompt, the API card, the repair prompts, paths	One place for every experimental setting (reproducibility rule)
`requirements.txt`	11	Pinned Python packages	Reproducible environment

`README.md`	55	Setup, liboqs DLL source and SHA-256	Others can rebuild the environment
`PROGRESS.md`	~600	Dated log of every step, decision, result and incident	Required by the brief; the audit trail
`.env`	–	`GROQ_API_KEY` (git-ignored)	Keeps the API key out of code and logs
`vendor/liboqs/bin/oqs.dll`	–	Prebuilt liboqs 0.16.0	liboqs-python cannot build it on this PC (no CMake/MSVC)
`docs/PROJECT_REPORT.md` / `.pdf`	–	This report	End-to-end documentation

14.2 Benchmark (`benchmark/`)

20 task folders (layout in Section 7.1), 157 Python files in total. `__pycache__` folders are generated and can be ignored.

14.3 The gate (`gate/`)

Path	Lines	What it does	Why
`gate/run_gate.py`	219	Runs the static tier (one Semgrep batch + dataflow) and the adversarial tier; outputs PASS / FAIL / ERROR as JSON or text; runs targets in a throwaway temp dir	Single entry point for checking any file
`gate/static_dataflow.py`	351	AST taint analysis: P1 (raw secret as cipher key) and P6 (hybrid KDF binds both secrets and the ciphertext)	Semgrep cannot express "something is missing along a data path"
`gate/adversarial.py`	545	Behavioural checks A1–A6; picks probes from the functions a module exposes so it works on any model output	Catches flaws that only show when the code runs
`gate/semgrep_rules/p1_…yaml`	33	Encap/decap output passed straight to AESGCM / ChaCha	Simple, direct form of F1
`gate/semgrep_rules/p2_…yaml`	68	RSA / EC / OAEP imports and calls	Classical crypto left behind (F3/F9)
`gate/semgrep_rules/p3_…yaml`	35	Secret-named values in log calls, print, raise, file writes	Key leaks (F5)
`gate/semgrep_rules/p4_…yaml`	11	Any use of the `random` module	Weak randomness (F6)
`gate/semgrep_rules/p5_…yaml`	26	mac/tag/check values compared with `==` / `!=`	Timing leaks (F8)

14.4 Harness (`harness/`)

Path	Lines	What it does	Why
`harness/migrate.py`	217	Sends the fixed prompt to each model (Ollama or Groq), saves the code and a full JSON log per sample; resumable; rate limits and network outages stop the run without writing anything	Phase 3 data generation
`harness/repair.py`	202	Phase 4 loop: feedback prompt (tests or tests + gate) → model → new file → tests + gate, up to 3 rounds; every round saved; resumable	RQ3

14.5 Scripts (`scripts/`)

Path	Lines	What it does	Phase
`smoke_test_oqs.py`	33	Checks ML-KEM and ML-DSA work	0
`smoke_test_ollama.py`	46	Checks both local models answer	0
`check_api_card.py`	20	Runs every line of the prompt's API card	0
`make_variants.py`	40	Builds `broken/` from each task's `variants.py`	1
`validate_benchmark.py`	79	All tests on all originals, references, variants	1
`benchmark_stats.py`	36	Variants per flaw and family	1
`validate_gate.py`	195	Gate + baselines (tests, default Semgrep, Bandit) on the benchmark; per-flaw and per-check tables	2
`evaluate.py`	192	Scores every model output: syntax, liboqs, tests, gate, baselines	3
`adversarial_worker.py`	30	Runs A1–A6 on one file in a separate process (so a hanging or crashing output cannot stop the evaluation)	3, 4
`keep_migrating.ps1`	43	Watchdog: relaunches migrations and the model download after crashes, rate limits and outages	3
`make_label_sample.py`	113	Picks the blind labelling sample, writes the guide and template, hides model names	3
`compare_labels.py`	94	Gate verdicts vs labels: precision, recall, per flaw, misses, false alarms	Checkpoint 3
`strict_migration.py`	90	Stricter "really migrated" test and headline numbers under it	Checkpoint 3
`keep_repairing.ps1`	25	Watchdog for the repair loop	4
`summarize_repair.py`	98	Fixed / broke tests / never changed per condition; per-check outcomes; list of fixes to audit	4

14.6 Data and results (`results/`, `labeling/`)

Path	Contents
`results/raw/<model>/<task>/sample_N.py/.json`	All 400 model outputs with full logs (prompt, raw response, model digest/version, timing)
`results/raw_superseded/`	Samples replaced because of infrastructure failures (kept for transparency)
`results/repair/<condition>/<model>/<task>/sample_N/round_K.py/.json`	Every repair round
`results/frozen/*.sha256`	Hashes of inputs and gate files before each run
`results/logs/`	Run logs (migrations, downloads, evaluation, repair)
`results/tables/benchmark_validation.csv`, `benchmark_flaw_counts.csv`	Phase 1
`results/tables/gate_validation.csv`, `gate_by_flaw.csv`, `gate_by_check.csv`, `gate_heldout_2026-10-14.json`	Phase 2

`results/tables/phase3_eval.csv`, `phase3_eval_strict.csv`	Phase 3 (one row per output)
`results/tables/gate_vs_labels_ai.csv`	Checkpoint 3
`results/tables/repair_items.csv`, `repair_summary.csv`	Phase 4
`labeling/GUIDE.md`, `labeling/items/`, `labeling/labels_ai.csv`, `labeling/labels.csv` (empty, for an optional human pass), `results/labeling_key.csv`	Blind labelling

15. How to reproduce

```
cd D:\Projects\Quantum_Encryption\pq-gate
.\.venv\Scripts\python.exe scripts\smoke_test_oqs.py
.\.venv\Scripts\python.exe scripts\validate_benchmark.py          # Phase 1
.\.venv\Scripts\python.exe scripts\validate_gate.py               # Phase 2
.\.venv\Scripts\python.exe harness\migrate.py                     # Phase 3 generation (resumable)
.\.venv\Scripts\python.exe scripts\evaluate.py                    # Phase 3 scoring
.\.venv\Scripts\python.exe scripts\make_label_sample.py           # labelling set
.\.venv\Scripts\python.exe scripts\compare_labels.py labeling\labels_ai.csv
.\.venv\Scripts\python.exe harness\repair.py                      # Phase 4 (resumable)
.\.venv\Scripts\python.exe scripts\summarize_repair.py
.\.venv\Scripts\python.exe gate\run_gate.py <file.py>             # check any single file
```

16. What remains

1. Write the paper (deadline 13 Nov 2026), using the corrected numbers and the limitations above.
2. Recommended: a human spot-check of about 10 of the AI labels in `labeling/labels_ai.csv`.
3. Optional: fix the gate bugs found (A2 naming, A5 coverage) as a new gate version, evaluated only on new data.